Senac Fecomércio Sesc

FACULDADE DE TECNOLOGIA E INOVAÇÃO SENAC-DF



ENGENHARIA DE SOFTWARE

Especificação de Requisitos - Documentação e Validação

ADS | Engenharia de Software | 2026

Prof. Ruben Prado

O que vamos construir hoje?

- A Ilusão da Pergunta Direta: Por que perguntar ao cliente "o que ele quer" geralmente falha?
- O que o caso do Milkshake nos ensina sobre requisitos reais.
- Como traduzir dores de negócio em especificações técnicas?
- Padrão de Documentação (DRS): Analisando o modelo do projeto TechEvent.
- A Abordagem Ágil: A transição de Requisitos Tradicionais para Histórias de Usuário (User Stories) e Critérios de Aceitação.
- Oficina Prática



O Paradoxo da Lista de Desejos

- Muitas equipes de software falham ao tentar simplesmente "dar aos clientes o que eles pedem". Um exemplo clássico ocorreu na Intuit (gigante de softwares financeiros): em um esforço bem-intencionado, eles pesquisaram exaustivamente seus clientes sobre quais recursos gostariam de ver. O resultado? Os clientes pediram 150 novos recursos.A equipe entrou em um modo de "busca de recursos", debatendo por semanas o que implementar. Na ausência de uma compreensão clara do motivo pelo qual os clientes usavam o produto, não havia bússola para priorizar. O viés de confirmação toma conta: a equipe de vendas foca no discurso comercial, a de marketing na marca, e a de P&D na tecnologia. Nenhuma reflete o verdadeiro trabalho que o cliente precisa realizar. Se você não sabe fazer a pergunta certa, não descobre a solução certa.



Você não compra um produto, você o contrata valor

O caso do Milkshake de uma rede de fast-food ilustra perfeitamente essa ideia. A rede queria vender mais milkshakes e perguntou aos clientes: "Querem mais barato? Mais chocolate? Mais espesso?". Eles mudaram a receita baseados no feedback, mas as vendas não subiram.

A virada ocorreu quando a equipe mudou a pergunta: "Que trabalho surge na vida das pessoas que as leva a 'contratar' um milkshake?". Descobriram que metade das vendas ocorria antes das 9h da manhã, por pessoas sozinhas, indo para o trabalho.

- **O Trabalho:** Manter a viagem interessante e evitar a fome no meio da manhã.
- **Os Concorrentes:** Bananas (acabam rápido e dão fome logo), Donuts (sujam o carro e as mãos) e Bagels (difíceis de passar cream cheese dirigindo). O milkshake vencia porque demorava 20 minutos para ser consumido e cabia no porta-copos. O requisito não era "mais chocolate", mas sim "durabilidade e praticidade no trânsito".



5 Pistas para encontrar o Requisitos

Como Analistas de Requisitos, precisamos investigar além do óbvio. O texto de inovação aponta 5 lugares onde as reais necessidades de software se escondem:

- **Dores pessoais** (Ex: Sal Khan criou a Khan Academy para ensinar matemática ao primo sem estresse).
- Quando não há solução e **o cliente simplesmente deixa** de fazer algo (Ex: 40% dos usuários do Airbnb não teriam viajado se a plataforma não existisse).
- Quando o **cliente faz "gambiarras"** para resolver algo. (Ex: Ligar para 5 restaurantes para achar uma mesa gerou o OpenTable).
- Coisas que as **pessoas odeiam fazer**. Eliminar esse atrito é um requisito de ouro (Ex: Filas de pronto-socorro geraram clínicas rápidas em farmácias).
- Quando o usuário usa seu sistema de um **jeito que você não planejou**, ele está gritando um requisito oculto para você.



Documento de Requisitos de Software

- Uma vez que entendemos o problema real, precisamos documentá-lo para que a equipe de desenvolvimento saiba o que construir. Para isso, utilizamos o DRS. No projeto de exemplo TechEvent (Plataforma Inovadora para Eventos), o documento estrutura a base do sistema dividindo-o em:
- **Escopo e Premissas:** O que o sistema vai fazer e o que NÃO vai fazer.
- **Requisitos Funcionais** (RF): Funções do sistema (Ex: RF-01 - O sistema deve permitir cadastro e autenticação de usuários).
- **Requisitos Não Funcionais** (RNF): Qualidade e restrições (Ex: RNF-01 - O tempo de resposta deve ser inferior a 3 segundos). O formato tradicional é vital para contratos, mas, para o desenvolvimento diário, o mercado de tecnologia migrou para uma abordagem mais ágil e centrada no usuário.



Foco no Valor, não na Máquina

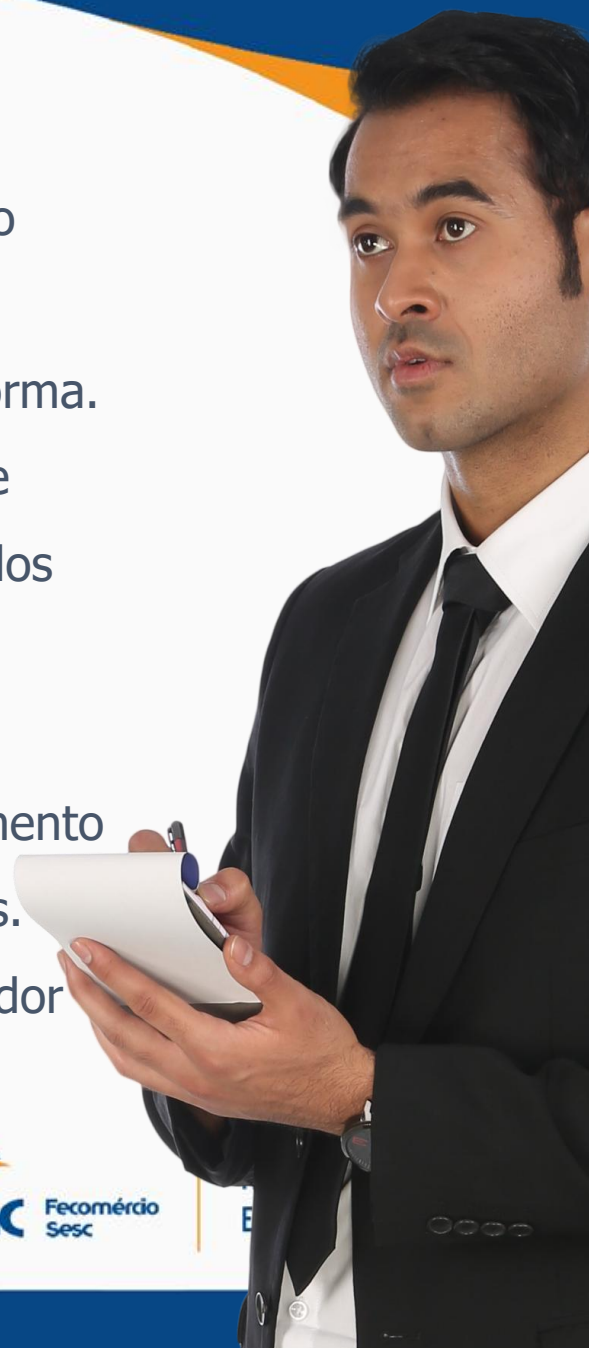
- Enquanto a documentação tradicional foca no que o sistema faz ("O sistema deve permitir check-in"), a documentação ágil foca em quem usa e por que usa. Chamamos isso de User Stories (Histórias de Usuário).
- A estrutura padrão e universal de uma História de Usuário é:
- **COMO [Ator/Perfil do usuário]**
- **EU QUERO [Ação ou funcionalidade que desejo]**
- **PARA QUE [O valor de negócio]**
- Essa estrutura força o Analista de Requisitos a sempre justificar o desenvolvimento de uma funcionalidade. Se você não consegue preencher o "PARA QUE", a funcionalidade provavelmente é inútil e não deveria ser desenvolvida.



Exemplo Prático: O Projeto TechEvent

- Vamos converter um requisito engessado em uma História de Usuário com base no nosso projeto TechEvent:
- **Formato Tradicional** (RF-01): Permitir que usuários realizem cadastro na plataforma.
- **Formato Ágil** (HU-01): COMO participante de eventos, EU QUERO me cadastrar e autenticar na plataforma PARA QUE eu possa me inscrever e acessar o ambiente dos eventos online.
- Outro exemplo (RF-10 Relatórios):
- **COMO** Administrador do Evento, **EU QUERO** visualizar um dashboard de engajamento **PARA QUE** eu possa analisar o sucesso do evento e melhorar as próximas edições. Notem como a intenção (o trabalho a ser feito) fica transparente para o programador que vai codificar a tela.





Como saber se a funcionalidade está "Pronta"?

A História de Usuário descreve o objetivo, mas os Critérios de Aceitação definem as regras e os limites técnicos. É a técnica de validação para garantir que desenvolvedor e cliente concordam sobre o que deve ser entregue. É o "contrato de teste".

Exemplo dos Critérios de Aceitação da HU-01 (Cadastro do TechEvent):

- O formulário deve exigir Nome, E-mail, CPF e Senha.
- A senha deve conter no mínimo 8 caracteres, incluindo letras e números.
- Se o CPF ou e-mail já constar no banco de dados, o sistema deve exibir a mensagem: "Usuário já cadastrado" e sugerir a recuperação de senha.
- Sem critérios de aceitação, o desenvolvedor pode criar uma senha de 3 caracteres e considerar a tarefa concluída, gerando bugs e retrabalho de segurança.



Não deixe os requisitos se perderem

Em sistemas complexos, é fácil esquecer por que uma tela existe ou perder o controle de quais requisitos já foram programados. Para evitar isso, utilizamos a Matriz de Rastreabilidade.

A Rastreabilidade é o mapeamento que liga o requisito original até a sua implementação final (código/tela). No projeto TechEvent, mapeamos assim:

1. **ID do Requisito: RF-01**
2. **História de Usuário: Cadastro de Usuário**
3. **Tela/Interface Correspondente: Tela_Login.html**

Se o cliente pedir para alterar a regra de cadastro (ex: adicionar login social com o Google), o Analista consulta a Matriz e sabe exatamente quais Histórias de Usuário e quais Telas do sistema serão impactadas pela mudança, permitindo estimar tempo e custo.





Senac Fecomércio Sesc

FACULDADE DE TECNOLOGIA
E INOVAÇÃO SENAC-DF